Inventory scanner import

SolariNet System of Record · Operator and developer guide · 21 October 2026

This guide covers the supported changeover from the standalone `scanner-ingest` app to the dashboard's authenticated SoR inventory flow. The scanner remains the portable capture tool; the dashboard is the preferred write path into the shared inventory System of Record. A restricted local CLI maintenance path remains available.

> **Release state:** The dashboard upload and import code and the RackWire model-profile editor are implemented in the integration worktree. JSX, PHP syntax, route authorization, UI behavior, scratch-MariaDB transaction tests, authenticated HTTP routes, and the scanner-to-RackWire browser acceptance pass locally. Apply is bound to the preview plan fingerprint and rejects a stale preview. No scanner export has been applied to the live SoR; staged dashboard deployment and live operator verification remain pending.

1. Operator procedure

1. In scanner-ingest, open **Export & backup** and choose **Download JSON** under Migration data. Keep the downloaded file private as an inventory export.
2. Open the live SolariNet dashboard and sign in with an operator account. Open **Inventory → Scanner import**.
3. Select the downloaded JSON file. The dashboard accepts schema version 1 exports up to 4 MiB and shows the catalog item and asset counts before sending anything.
4. Review location matching. A unique byte-for-byte name match can be used automatically. The preview lists every source location and its resolved SoR ID and name. For duplicates or renamed locations, select the intended SoR location by its visible ID and name. Case or accent differences require an explicit mapping. For an unmatched location, either stop and fix the mapping or explicitly choose to preserve its source text in notes with no SoR location assigned.
5. Optionally match a scanner catalog item to an existing SoR model. Leave it unmatched to add or reuse the exact make/model. Optionally expand category mapping to override the automatic SoR hardware type for new models; categories that exactly match a type ignoring case map automatically, and unknown categories default to `other`. A category override does not change a model you matched to an existing SoR row. Review any assets marked `repair`; choose whether they should enter SoR as `in_stock` or `retired`. No repair default is selected.
6. Choose **Preview import**. Check the insert/reuse/skip counts and warnings. Preview performs reads only and writes no inventory.
7. Choose **Apply import** after checking the preview. Apply replans against the current SoR state and compares a plan fingerprint with the preview. If relevant inventory state changed, apply stops and asks you to preview again. A matching plan commits in one transaction or rolls back. On success, the dashboard refreshes its inventory view.

8. To place an imported device in a connection plan, open **Inventory → Components → RackWire device profiles**. Select its SoR model, define connector groups, port counts, link speeds, PoE capability and budget, and device/port power values supported by the hardware, then save. Confirm the hardware specifications before using the profile for design decisions. Open RackWire and place the model; scan its barcode to resolve the unit from shared inventory.

> **Location identity matters.** The SoR may contain more than one location with the same display name. Select by location ID after confirming the physical destination. Do not map a scanner label to a similarly named location by guesswork.

Realtime network discovery and host ingest

For network-attached systems, open **Discovery** in the dashboard, enter an authorized IPv4 range, and start a bounded scan (up to 4,096 addresses). SolariNet queues the scan and returns immediately while the native worker probes hosts. The page polls job progress every second and refreshes saved candidates every 2.5 seconds while the worker runs, so reachable hosts can be reviewed before the scan finishes. Review each candidate, correct its name or class where needed, then choose **Adopt** to add it to monitoring. Adoption also mirrors the host name and IP address into the SoR network entity/address records, where normal SoR synchronization can publish supported network facts.

> **Keep the two ingest paths distinct.** Network discovery identifies and monitors reachable hosts. Adoption does not create a physical `hardware_units` asset, barcode, purchase/location record, or verified RackWire model profile. Use scanner-ingest import or the inventory editor for physical equipment; then scan its barcode in RackWire to link the planned device to the SoR asset. The authenticated browser flow verifies candidate refresh before scan completion using a controlled fixture; a separate native-worker acceptance scans a loopback listener and verifies its candidate in Discovery. A full lab-subnet scan and deployed worker behavior remain unverified.

> **Live mapping check · read-only · 21 October 2026.** The scanner source has 2 catalog items and 2 assets, with one asset at `Bin #5` and one at `Nook`. The live SoR has two active location rows, IDs `9103` and `9104`, both named `Painted Fern`; a direct live query confirms both are root-level `site` rows with no parent path to distinguish them. Neither scanner label has a byte-exact match. Map each by physical destination before applying, or explicitly keep its source label in notes with no SoR location. No inventory rows were written by this check.

2. Import behavior

Scanner data	SoR behavior	Operator consequence
Catalog item	Reuses an existing exact make/model or inserts a hardware model. The scanner UUID is retained in `external_refs`; original item fields are preserved in notes.	Use an explicit model match only when the destination model is the same product.
Physical asset	Inserts one `hardware_units` row with its tag, serial, mapped location, quantity 1, stock state, and scanner provenance.	An existing asset tag/serial collision stops the entire transaction for reconciliation.
Asset tag and serial	Creates per-unit `barcodes` records where values are present. Original scanner details remain in notes.	A code already assigned elsewhere is a conflict; no code or unit is overwritten.
Archived source rows	Archived catalog items and assets are skipped. Active assets cannot refer to an archived item.	Restore or correct source records in scanner-ingest before export if they should be active.
Repair status	Requires an explicit choice of `in_stock` or `retired`.	Resolve physical condition before applying; SoR has no repair stock state.

Each imported fact is stamped with the `scanner-ingest` source and a dashboard operator audit entry. Repeating the same export is idempotent: unchanged rows are reused. If a previously imported row differs from its SoR state, the import stops and leaves existing data unchanged. Reconcile the discrepancy in the SoR or correct the source record before retrying.

3. Failure and recovery

Message or condition	Action
Wrong schema, malformed JSON, or file too large	Export JSON again from scanner-ingest. The importer accepts only schema version 1 and caps the upload at 4 MiB.
No unique SoR location / ambiguous location	Choose a destination by ID, or deliberately preserve the source label in notes. Verify the physical room or bin first.
Repair status needs a decision	Inspect the asset and choose `in_stock` or `retired` before previewing again.
Existing inventory collision or changed prior import	Stop and reconcile the matching asset/model in the SoR. The importer is insert-only and does not repair existing rows.
SoR unavailable or transaction failed	No partial import is committed. Restore dashboard-to-SoR availability, then preview again.

The importer serializes concurrent applies with a MariaDB advisory lock shared with the standalone CLI. The lock is released after success or failure. The dashboard session, operator role, and CSRF policy protect both preview and apply routes. The standalone scanner app remains loopback-bound by default and does not need to be exposed to the LAN.

Preview returns a SHA-256 fingerprint over the normalized insert plan, counts, warnings, and resolved locations. Apply recomputes it after acquiring the shared import lock and refuses a mismatch. The database timestamp is excluded from the fingerprint because it is generated when rows are inserted.

4. Developer contract

- `dashboard/api/lib/ScannerImport.php` validates the portable snapshot, inspects destination schema and enum constraints, builds bounded lookup indexes for the fields it matches, confirms candidates with SoR collation rules, and executes one insert-only plan transactionally. Preview and apply share the planner.
- `POST /api/inventory/scanner-import/preview` and `POST /api/inventory/scanner-import/apply` are operator routes registered through the existing inventory policy. Payload: `{snapshot, mapping, repairStatus, unmappedLocation}`; apply also requires the preview's 64-character `expectedFingerprint`.
- Supported mapping keys are `locations` (scanner label → SoR location ID), `models` (scanner item UUID → SoR model ID), and `categories` (scanner category → SoR hardware type). The UI exposes all three. Category matching defaults to a case-insensitive exact hardware-type name, then `other`; an explicit category choice only affects newly created models.
- Snapshot bounds: at most 1,000 items and 5,000 assets; request body at most 4 MiB. The byte cap accommodates realistic exports at the full row limit while remaining below PHP's standard 8 MiB post body limit. Source IDs must be canonical lowercase UUIDs, matching scanner-ingest

exports; uppercase variants are rejected because SoR external-reference keys use a case-insensitive collation. Archived state and version are validated before DB access.

- Destination tables must use InnoDB and match the required columns from SoR migrations 014 and 015, including `external_refs`, `barcodes`, and `audit_log`. Length, enum, soft-delete, collation, reference, and uniqueness checks fail closed.
- The scanner app's JSON export is the interface. No direct browser call to its unauthenticated local API, remote DB credential, LAN listener, or server-side JavaScript is introduced.

Restricted local CLI maintenance path

`scanner-ingest/export_sor.py` remains available for a named operator on a controlled host when the dashboard path cannot be used. It connects directly to MariaDB and is not the default changeover path. Use a restricted database account, the verified SoR TLS CA, and an approved import window. The CLI preview is read-only; applying requires the explicit `--apply` flag and a single transaction. It does not grant access to the scanner's LAN service or copy its database credentials into the dashboard.

- It reads SQLite in read-only/query-only mode and imports only active catalog items and assets. Archived rows are skipped; it does not infer SoR deletion or archive actions.
- CLI preview returns a plan fingerprint; `--apply` requires that fingerprint and rejects a changed plan. The dashboard and CLI share the same MariaDB advisory lock.
- Use an explicit mapping file for scanner locations, model IDs, or categories that do not match the SoR. Repair state needs an explicit `--repair-status in_stock` or `--repair-status retired`; unmatched locations stop unless `--unmapped-location notes` is deliberately selected.
- UUID references make repeated unchanged imports idempotent. Existing inventory is never overwritten; changed source rows, barcode collisions, soft-deleted references, and destination length violations stop for reconciliation.
- `scanner-ingest/sor-importer-role.sql` defines the CLI role: SELECT on `sources`, `hardware_models`, `hardware_units`, `locations`, `external_refs`, and `barcodes`; INSERT on those write tables plus `audit_log`. It grants no UPDATE, DELETE, DDL, or audit-log reads. Provision a dedicated account with a host restriction and protected secret outside Git; do not reuse the broad SoR synchronization account.
- Live SoR preview on 21 October 2026 found an insert plan for two catalog items and two assets. Both source location names were deliberately retained only in notes because neither matched a live SoR location. Preview was read-only. The available synchronization account has broader privileges than the CLI needs, so no live apply or role provisioning has been performed.
- The role definition and preview/apply/replay path passed all 15 database-backed exporter tests on disposable MariaDB 11.8.6, including denial checks for UPDATE and DELETE. The tests must never target production.

See `scanner-ingest/EXPORT-SOR.md` for the full command contract and field mapping. Keep the dashboard flow as the normal operator path after its release is deployed.

Verification and release gate

Check	Current evidence
Dashboard JSX parse	**PASSED** Vendored Babel parses all 16 dashboard JSX files, including `screens4.jsx`.
PHP syntax and route authorization	**PASSED** Import class and route file lint clean; route audit reports all writes operator/admin gated.
Scanner importer and duplicate-name UI	**PASSED · 21 OCT 2026** `tests/dashboard/test_scanner_import.php` ran against a temporary local MariaDB 11.8 database using SoR migrations 001, 014, 015, and 020. It covers preview-only behavior, byte-exact location matching, rejection of uppercase source UUIDs, stale-plan rejection after concurrent SoR change, apply, repeat import, no-overwrite conflict, and full rollback on collision; fixture rows were removed. `tests/dashboard/test_scanner_import_ui.js` uses two same-name location rows (#9103 and #9104), verifies both IDs are visible, selects #9104, and checks that preview sends and displays that exact ID. It also chooses a category override and verifies the request carries that SoR hardware type. The standalone scanner-ingest suite ran 63 tests: all passed, including the optional ZXing decoder and all 15 MariaDB exporter cases against a disposable MariaDB 11.8.6 schema. On 21 Oct 2026, the dashboard import planner, UI, authenticated HTTP routes, and browser flow also passed against separate migrated loopback SoR fixtures. The browser test uploads a schema-conformant export fixture. Additionally, the running local scanner app's actual schema-v1 JSON export (2 items, 2 assets) was exercised against a fresh disposable SoR with explicit location mappings: preview left all inventory counts unchanged; apply created 2 models, 2 units, 4 external references, and 2 barcodes; an identical preview-bound replay created no duplicates, and both location IDs read back. The disposable database was stopped and its port closed. No live SoR write occurred. The production dashboard remains undeployed and unverified.
Current SoR inventory scale	**OBSERVED · 21 OCT 2026** A read-only transaction through the deployed SoR service account returned aggregate counts only: 37 active hardware models, 40 active physical units, and zero active receipts, projects, or project allocations. No inventory rows or credentials were emitted. **Inference:** the combined inventory read is modest at the current scale; revisit pagination if the catalog or asset count grows materially.
Authenticated scanner HTTP routes	**PASSED · 21 OCT 2026** `tests/dashboard/test_scanner_import_http.php` exercised the real PHP front controller over loopback against a disposable MariaDB 11.8 schema. Both preview and apply reject anonymous and viewer sessions; missing CSRF is rejected on both endpoints, and apply requires the preview fingerprint. Valid preview/apply passed, including an explicit category-to-hardware-type override; stale fingerprint conflicted; a fresh preview plus apply replayed idempotently. The suite also verifies 5,000-asset preview fits under the 4 MiB request limit without writing inventory. Temporary inventory rows and the database container were removed. The check is wired into dashboard CI.

Check	Current evidence
Scanner, SoR, RackWire, and live Discovery UI	**PASSED · 21 OCT 2026** `tests/browser/test_scanner_import_e2e.cjs` ran through the actual dashboard in Chromium against a fresh loopback SoR schema. It created a RackWire device profile, resolved a barcode to the imported SoR asset, persisted the device plan, submitted an asynchronous scan, and verified progress survived navigation. A controlled candidate then appeared in Discovery while the job was still running, before the fixture marked the scan complete. The SoR fixture, inventory and plan rows, and temporary browser session files were cleaned up. No production data was involved.
Native scan worker to Discovery candidate	**PASSED · 21 OCT 2026** `tests/browser/test_scan_worker_browser_e2e.cjs` ran the native worker, C bridge, authenticated dashboard, and a fresh SoR/monitoring database. It scanned a loopback listener, observed progress through navigation away and back, and verified the saved candidate in Discovery. The scan was limited to `127.0.0.1/32`; the scratch databases and container were removed. No production network or data was involved.
Deployed dashboard / live SoR verification	**UNVERIFIED** The browser acceptance used the local candidate and a disposable database. A staged dashboard deployment and operator-session verification against the intended SoR remain pending before production changeover.

Run the database-backed tests (`test_sor_transactions.php`, `test_scanner_import.php`, `test_scanner_import_http.php`, and `test_scanner_import_e2e.cjs`) only against a disposable local database named `solarinet_*`, with `SOLARI_TEST_SOR_DB=1` and loopback `SOR_DB_HOST`. Each harness refuses a non-loopback host or non-scratch database name. The HTTP and browser tests start private local services and synthetic operator sessions; none reads or writes the live SoR. The browser acceptance also removes its temporary database user and schema.

5. UI style decisions

The import is embedded in Inventory and follows the dashboard’s existing dark, compact administrative UI. The workflow has one primary action at each stage: select the file, review mappings, preview, then apply. Advanced model and category mapping are collapsed by default. The category section states the automatic mapping and fallback; location IDs stay visible where duplicate names need distinction. Errors are announced in an alert region, asynchronous results use status text, controls remain keyboard reachable, and no motion is required to understand the state.